

Property-Based Testing

Prof. Dr. Raphael Herding

Why is testing hard?

Example-based Testing

- set up one input scenario
- run the code under test
- check the output
(and any other effects the code is supposed to have)

**Imagine we needed to
implement a custom
ADD function**

```
@Test
public void testSum() {
    assertThat(sum(1, 2), equalTo(3));
}
```

```
private int sum(int a, int b) {  
    if (a == 1 && b == 2) {  
        return 3;  
    } else {  
        return 0;  
    }  
}
```

@Test

public void testSum() {
 assertThat(sum(2, 2), *equalTo*(4));
}

```
private int sum(int a, int b) {  
    if (a == 1 && b == 2) {  
        return 3;  
    } else if (a == 2 && b == 2) {  
        return 4;  
    } else {  
        return 0;  
    }  
}
```


Enterprise Developer



```
@Test
public void testSum3() {
    // When I add two numbers I expect their sum
    for (int i = 0; i < 100; i++) {
        int x = randomInt();
        int y = randomInt();
        assertThat(sum(x, y), equalTo(x + y));
    }
}
```

```
@Test
public void testSumUsingCommutativityProperty() {
    // Changing the order of arguments
    // shouldn't change their sum
    for (int i = 0; i < 100; i++) {
        int x = randomInt();
        int y = randomInt();
        assertEquals(sum(x, y), sum(y, x));
    }
}
```

// unfortunately that's not enough

```
private int sum(int a,    int b) {  
    return a * b;  
}
```

@Test

// `x + 2` is the same as `x + 1 + 1`

```
public void testSumWithAdditiveProperty() {  
    for (int i = 0; i < 100; i++) {  
        int x = randomInt();  
        int result1 = sum(sum(x , 1), 1);  
        int result2 = sum(x, 2);  
        assertEquals(result1, result2);  
    }  
}
```

```
// WTF?!  
private int sum(int a, int b) {  
    return 0;  
}
```

Testing Specification with properties

- Commutativity property
- Associativity property
- Identity property

Testing Specification with properties

By using specifications, we understood the
requirements in a much deeper way

Lab

- **pb-01:** Shift from example-based thinking to property-based thinking: define what must always be true, not just what one result should be

What is Property-Based Testing?

A lightweight tool for random testing of programs

PBT = Random Generation + Executable Specifications

What is Property-Based Testing?

A lightweight tool for random testing of programs

$$\text{PBT} = \text{Random Generation} + \text{Properties}$$


Example Based Test

Example based tests are executable specifications over specific inputs to programs.

```
def test_append():  
    l = [1, 2, 3]  
    l.append(4)  
    assert l == [1, 2, 3, 4]
```

*all examples will be using Python and Hypothesis

```
def test_append():
```

```
    l = [1, 2, 3]
```

```
    l.append(4)
```

```
    assert l == [1, 2, 3, 4]
```

```
def test_append2():
```

```
    l = [1, 2, 3]
```

```
    l.append(4)
```

```
    assert 4 in l
```

```
@given(lists(integers()))
```

```
def test_append3(l:
```

```
list[int]):
```

```
    l.append(4)
```

```
    assert 4 in l
```

```
@given(lists(integers()), integers())
```

```
def test_append4(l: list[int], x):
```

```
    l.append(x)
```

```
    assert x in l
```

Example Based Test

`append([1, 2, 3], 4) = [1, 2, 3, 4]`

Property Based Tests

List contains the appended element.

$\forall l, x. x \text{ in } \text{append}(l, x)$

Last element of the list is the appended element.

$\forall l, x. \text{append}(l, x)[-1] = x$

Prefix of the list does not change.

$\forall l, x. \text{append}(l, x)[: -1] = l$

List length increases by 1.

$\forall l, x. \text{len}(\text{append}(l, x)) = \text{len}(l) + 1$

```
def append(l: list, x: any):  
    l = l.copy()  
    l.append(x)  
    return l
```

```
@given(lists(), integers())  
def test_contains(l: list, x):  
    assert x in append(l, x)
```

Property

A property is an executable specification of a program over an abstract set of inputs.

Sorting is idempotent.

$\forall l. \text{sorted}(\text{sorted}(l)) = \text{sorted}(l)$

Reversing a list 2 times results in the original list.

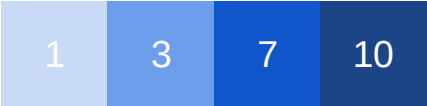
$\forall l. \text{list}(\text{reversed}(\text{list}(\text{reversed}(l)))) = l$

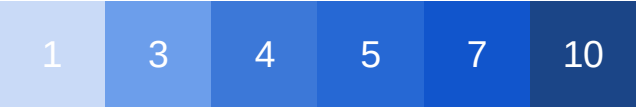
Types of Properties

Validity Testing

Every operation should return valid results.



`delete(`  `,`  `)` `=` 

`insert(`  `,`  `)` `=` 

Every insert/delete on a sorted list should result in a sorted list.

```
@given(sorted_lists(), integers())  
def test_sorting_validity(sl, x):  
    assert sl.insert(x).is_sorted()  
    assert sl.delete(x).is_sorted()
```

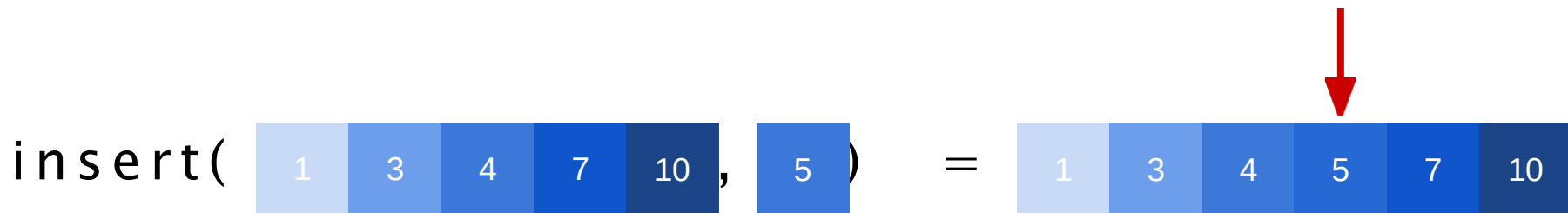

Types of Properties

Validity Testing

Every operation should return valid results.

Postcondition Testing

The predicate should hold after any occurrence of the operation.



Inserted value should be in the list.

```
@given(sorted_lists(), integers())
def test_sorting_postcondition(sl: SortedList, x: int):
    assert x in sl.insert(x)
```

Types of Properties

Validity Testing

Every operation should return valid results.

Postcondition Testing

The predicate should hold after any occurrence of the operation.

Metamorphic Testing

Two operations should relate to each other.

`insert(insert( [ 1 3 4 7 10 ], [ 2 ] ), [ 5 ]) = insert(insert( [ 1 3 4 7 10 ], [ 5 ] ), [ 2 ])`

Insertion order should be irrelevant.

```
@given(sorted_lists(), integers(), integers())
def test_sorting_metamorphic(sl: SortedList, x: int, y: int):
    assert sl.insert(x).insert(y) == sl.insert(y).insert(x)
```

Types of Properties

Validity Testing

Every operation should return valid results.

Metamorphic Testing

Two operations should relate to each other.

last() =

first() =

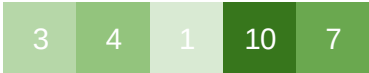
find( , ) =

Postcondition Testing

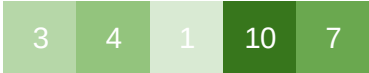
The predicate should hold after any occurrence of the operation.

Model Testing

The implementation should conform to the model.

max()

min()

find( , )

Lab

- **pb-04:** Use *@composite* to generate realistic domain objects and verify business invariants that no handwritten example set would cover.

```
@given(lists(integers()), integers())
def test_sorting_model(l: list[int], x: int):
    sl = SortedList(l)
    if len(l) == 0:
        assert sl.last() == max(l)
        assert sl.first() == min(l)
        assert sl.find(x) == (x in l)
    else:
        assert len(sl) == 0
```